

Zera for Feather

Representative Agent Trajectories

A sanitized, human-readable record of the agent instructions, observable actions, tool responses, retries, human checkpoints, and final outcomes that shaped Zera's rubric validation workflow.

Evidence status

These trajectories are reconstructed from observed project runs, captured UI failure cases, the current worker instructions, and verified implementation behavior. They are representative traces rather than raw provider logs. Secrets, credentials, cookies, private chain-of-thought, and personal data are intentionally excluded.

Project	Zera for Feather
Primary user	Rubric authors preparing bug and feature-request batches for submission
Live application	zera-rubric-validator.vercel.app
Repository	github.com/aman-codes-1/zera-rubric-validator
Prepared	31 August 2026
Agents covered	Documentation review agent; correction-repair agent; Codex development agent

Submission purpose

The hackathon asks for representative trajectories for every agent used. This artifact makes each trajectory auditable from instruction to final result, including tool behavior, feedback, retries, and checkpoints.

01 Scope and agent inventory

What is traced, what is intentionally excluded, and how the agents connect.

What counts as a trajectory

A trajectory is the observable sequence from an agent's assigned goal to a usable result. It includes the input, instructions, actions, tool calls, summarized tool responses, next-step decisions, retries, and human checkpoints. It is not a software stack trace and it does not require hidden reasoning.

AGENT / ROLE	PURPOSE	TOOLS	OUTPUT
Documentation review agent	Grammar-check every text field and evaluate feature-request capabilities against official product documentation.	OpenAI Responses API; web search; structured JSON schema	Per-rubric documentation status, findings, exact excerpts, corrected fields, and sources
Correction-repair agent	Repair a warning or error when its corrected field is missing or repeats the original value.	OpenAI Responses API; strict structured output; no new web search	Distinct full-field correctedText values synchronized into correctedRubric
Codex development agent	Translate human feedback and failure screenshots into implementation changes and verification.	Repository inspection; patches; build and type checks	Working UI and worker changes, README updates, and verified builds

End-to-end handoff

1	2	3	4	5
User supplies application, batch, and JSON	Deterministic checks establish eligibility	Review agent researches and returns findings	Repair agent fixes incomplete corrections when needed	User reviews evidence and copies corrected JSON

Trace boundary

The tables show observable actions and concise decision summaries. They do not disclose private chain-of-thought. All API keys and authentication material are excluded.

02 Trajectory 1 - Documentation review agent

Representative challenging case: QuickBooks feature-request capability is only partially supported by retrieved documentation.

Goal and input

Goal	Review a complete rubric batch and return one structured review per rubric.
Application	quickbooks
Rubric type	feature request
Affected field	expected_behavior
Observed input	The "More actions" menu should be enabled on the "BANK ACCOUNTS", "PROFIT & LOSS", and "EXPENSES" cards and should open the available card-specific actions when selected.

Relevant agent instructions

- Use web search only for feature-request rubrics and prefer official vendor documentation.
- Evaluate only the functional capability described in criterion, expected_behavior, and actual_behavior.
- Do not validate section names, navigation paths, UI labels, placement, or locations against documentation.
- Check every text field for grammar and clarity, but keep tags and deterministic format rules outside documentation review.
- Return exact excerpts, line numbers, findings, correctedText, correctedRubric, and one review for every supplied index.

Observable execution

STEP	AGENT ACTION	TOOL OR SYSTEM RESPONSE	NEXT DECISION
1	Receive batch	The client sends application, batch preset, and the full rubric JSON array to POST /api/validate.	Treat rubric content as data and begin the structured review.
2	Apply scope rules	The rubric has the single tag feature request. Documentation review applies only to criterion and behavior fields.	Exclude page_or_workflow, reproduction_steps, tag inference, and UI placement from documentation judgment.
3	Search official sources	Web search returns official QuickBooks material supporting Home widgets and dashboard-style insights.	Compare only the claimed capability with the supported high-level behavior.
4	Assess evidence	The retrieved material does not confirm a card-level More actions menu on every named card.	Set documentationStatus to unclear rather than inventing product details.
5	Create finding	The agent returns a warning on expected_behavior with line 1, the exact affected excerpt, and a concrete recommendation.	Keep the rubric usable but require a more defensible capability statement.
6	Return structured result	The response conforms to the strict JSON schema and preserves the rubric index.	Allow the client to combine this evidence with deterministic findings.

03 Trajectory 1 - Result and evidence

What the user sees, how the finding changes status, and where the output remains auditable.

Representative documentation response

Documentation status: unclear	
Official QuickBooks Home documentation supports the "Business at a glance" section, but the reviewed sources do not confirm a card-level "More actions" menu for every listed card.	
Severity	warning
Field	expected_behavior
Line	1
Finding	The documentation supports Home widgets and dashboard-style insights, but not the exact presence of a "More actions" menu on each named card. The claim is only partially supported.
Recommended change	Specify a documented card and action, or frame the statement as a feature request at the supported level of specificity.

Status propagation

- The affected phrase is highlighted in yellow in the source field.
- The rubric becomes Passed with warnings rather than Valid.
- The batch warning count and Needs fix count increase.
- The documentation summary and source links remain visible for review.
- The corrected replacement is included in the per-rubric preview and top-level copied batch when it differs from the original.

Human checkpoint
The user decides whether the warning is acceptable, whether stronger product evidence should be provided, or whether the corrected JSON should be submitted. Zera does not silently convert uncertainty into a pass.

Traceability evidence

EVIDENCE	WHERE IT APPEARS	WHY IT MATTERS
Exact input field	Expanded rubric card	Connects the finding to the user's original text.
Documentation summary	Documentation review panel	Makes the evidence boundary visible.
Finding and line number	Findings and corrections panel	Shows the exact reason and affected location.
Source URLs	Documentation sources section	Lets a reviewer verify the retrieved evidence.
Corrected JSON	Per-rubric and batch copy actions	Produces a usable artifact rather than advice alone.

04 Trajectory 2 - Correction-repair agent

Retry path triggered when a completed review contains a missing correction or repeats the original field unchanged.

Trigger

The main review is complete and its finding is valid, but correctedText is empty or character-for-character equal to the original field. Earlier versions either displayed the unchanged value as a correction or rejected the whole review. Human feedback required a third behavior: repair the correction without blocking the completed evaluation.

Human feedback that changed the path

"Do not block" was followed by "provide corrected rubric for each warning or error." The implementation therefore treats correction fidelity as a repairable output problem, separate from review validity.

Repair instructions

- Preserve applicationBrief, indexes, documentation status and summary, findings, severities, messages, suggestions, line numbers, and excerpts.
- Return a complete rewritten value for every warning and error.
- The replacement must visibly differ from the original and must implement the finding.
- Copy correctedText exactly into the corresponding correctedRubric field.
- Do not perform a new web search or invent undocumented product details.

Observable execution

STEP	AGENT ACTION	TOOL OR SYSTEM RESPONSE	NEXT DECISION
1	Compare values	The client compares finding.correctedText and correctedRubric field values with the original field.	Detect missing or unchanged corrections.
2	Start repair	The client sends action=repair_corrections with the original rubrics and completed review.	Keep the main result available; repair only the correction payload.
3	Preserve evidence	The repair agent receives the existing findings and instructions not to change evidence or severity.	Rewrite the affected field without re-running documentation research.
4	Generate replacement	A defensible replacement narrows the claim: "A More actions control should be available on supported Business at a glance cards and should open the documented actions for the selected card."	Synchronize the same value into correctedRubric.expected_behavior.
5	Return and poll	The background response is polled without a fixed deadline until completion or an explicit API error.	Merge the repaired review while preserving the original documentation sources.
6	Render and export	The corrected field appears in green and is reapplied by both per-rubric and batch copy actions.	Deliver a correction that is visibly different and usable.

05 Trajectory 3 - Codex development agent

Representative build-time trajectory showing human feedback, failed assumptions, code changes, and verification.

Assigned goal

Turn the observed mismatch between the original rubric and its displayed correction into a reliable product behavior, while preserving the user's requirement that validation must not be blocked solely by an unchanged correction.

Observable execution

STEP	AGENT ACTION	TOOL OR SYSTEM RESPONSE	NEXT DECISION
1	Receive failure evidence	Human supplies a screenshot and states: "there is no difference between original and corrected."	Inspect the correction selection and export paths.
2	Search repository	Code search locates <code>correctedTextForFinding</code> , <code>applyFindingCorrections</code> , <code>locateFinding</code> , <code>report rendering</code> , and <code>worker instructions</code> .	Identify an unconditional fallback that can relabel unchanged text as corrected.
3	Apply first fix	The display begins rejecting unchanged replacements and the client initially fails closed when the response is inconsistent.	Run a production build and verify TypeScript compilation.
4	Receive human checkpoint	Human explicitly rejects the blocking behavior: "do not block."	Remove the validation failure while continuing to hide fake corrections.
5	Receive second checkpoint	Human reports: "no corrected rubric, provide corrected rubric for each warning or error."	Add a correction-only repair request rather than restoring the misleading fallback.
6	Patch client and worker	Client detects weak corrections, starts repair, polls, preserves sources, and merges output. Worker adds strict repair instructions and schema output.	Verify formatting and production compilation.
7	Update documentation	README gains Iteration 9, regression evidence, validation-flow details, Codex usage, and the live app link.	Keep the implementation story aligned with observed evidence.

Verification evidence

CHECK	RESULT	SIGNIFICANCE
Production build	Next.js compilation and TypeScript checks passed after the correction changes.	Confirms that the client and worker integration compiles as a complete application.
Diff formatting	git diff --check returned no whitespace errors.	Confirms the patch is clean for source control and review.
Human acceptance	Blocking behavior was removed after direct feedback; correction repair was added after the missing-correction report.	Shows feedback shaping the agent's next step rather than being ignored.

06 Retries, feedback, and human checkpoints

The moments where uncertainty or failure changed the workflow.

SIGNAL	AGENT RESPONSE	CONTROL / LEARNING
Main API returns 404, HTML, quota failure, or incomplete data	Clear results and show validation unavailable instead of presenting local checks as a pass.	A visible failure is safer than a false green report.
Documentation supports only a high-level capability	Return unclear plus a field-specific warning rather than claiming support or failure with false certainty.	Uncertainty remains actionable and visible.
Documentation is not found for a feature request	Return a field-specific documentation error that contributes to Errors and Needs fix.	Evidence changes validity; red documentation cannot coexist with a Valid result.
Bug rubric has no supporting documentation	Skip documentation review and run grammar review only.	A bug report is an observation; missing documentation is not evidence that the bug is invalid.
Correction equals original	Run the correction-repair agent without invalidating the completed review.	Correction quality is repaired independently from review validity.
User reviews final warnings or errors	Expose evidence, exact affected text, recommendation, correction, and source links before copy or submission.	The intended user remains the final checkpoint.

Security and privacy controls

- OPENAI_API_KEY remains server-side and is never included in the trace.
- Authorization headers, cookies, access tokens, and response identifiers are omitted.
- Rubric examples use product UI descriptions and contain no personal data.
- Tool responses are summarized; full copyrighted documentation is not reproduced.
- Only observable actions and concise decision summaries are shown; private chain-of-thought is excluded.

07 Evidence index and submission checklist

How a judge can connect this artifact to the repository and reproduce the main workflow.

Repository evidence

PATH	EVIDENCE
worker/feather-worker.js	Main review instructions, web-search configuration, strict response schema, correction-repair instructions, background execution, and response retrieval.
app/page.tsx	Input gates, deterministic checks, polling, repair detection, report rendering, exact highlighting, status calculation, and corrected JSON copy behavior.
scripts/dev-server.mjs	Local server bridge that keeps the OpenAI key outside client code and exposes the validation endpoint during development.
public/sample-rubrics.json	Reproducible example batch for the QuickBooks workflow.
README.md	Problem statement, user value, improvement changelog, regression evidence, failure mode, hot take, setup, and validation behavior.

Submission checklist

- **CHECK** Attach or link this PDF in the hackathon submission.
- **CHECK** Link the GitHub repository and live application.
- **CHECK** Keep the full agent instructions in the repository.
- **CHECK** Include one realistic end-to-end run in the solution video.
- **CHECK** Include baseline-versus-final evaluation results separately; trajectories are not a replacement for test cases.
- **CHECK** If raw API exports are available, add sanitized copies beside this PDF as supplemental evidence.

Main takeaway

Zera's trajectories show a controlled agentic workflow: deterministic rules decide when review may begin, the documentation agent handles semantic uncertainty, the repair agent fixes weak correction payloads, and the human remains the final checkpoint before submission.

Prepared as a representative, sanitized trajectory artifact for the micro1 Agentic Workflows Hackathon.